

Main.java:

```
//TIP To <b>Run</b> code, press <shortcut actionId="Run"/> or
// click the <icon src="AllIcons.Actions.Execute"/> icon in the gutter.
public class Main
{
    public static void main(String[] args)
    {
        //create a new graph
        Graph g = new Graph();

        //add vertices (tourist sites) to the graph
        g.addVertex("Train Station", 4, 9);
        g.addVertex("Library", 3, 7);
        g.addVertex("Market Square", 2, 4);
        g.addVertex("St. Michael's Church", 1, 6);
        g.addVertex("Leisure Centre", 6, 5);
        g.addVertex("Peoples Park", 5, 3);
        g.addVertex("Solas Eco Garden Centre", 4, 2);

        //insert edges (walking paths) between the sites
        g.Insert("Leisure Centre", "Market Square", 0.9);
        g.Insert("Peoples Park", "Leisure Centre", 1.2);
        g.Insert("St. Michael's Church", "Peoples Park", 1.6);
        g.Insert("Train Station", "Library", 1.8);
        g.Insert("Library", "St. Michael's Church", 2.0);
        g.Insert("Market Square", "Solas Eco Garden Centre", 2.3);

        //test for the implemented algorithms with sample data
        System.out.println();
        g.Search("Library");

        System.out.println();
        g.AllCons("Leisure Centre");

        System.out.println();
        g.Closest("Peoples Park");
    }
}
```

Vertex.java:

```
public class Vertex
{
    //Attributes for Vertex class
    String name;
    public int x;
    public int y;

    //Constructor for Vertex class
    public Vertex(String name, int x, int y)
    {
        this.name = name;
        this.x = x;
        this.y = y;
    }
}
```

Edge.java:

```
public class Edge
{
    //Attributes for Edge class
    String start;
    String end;
    double weight;

    //Constructor for Edge class
    Edge (String start, String end, double weight)
    {
        this.start = start;
        this.end = end;
        this.weight = weight;
    }
}
```

Graph.java:

```
public class Graph
{
    //Attributes for Graph class
    Vertex[] VertexList = new Vertex[20];    // max 20 vertices (tourist
    sites)
    Edge[] EdgeList = new Edge[40];          // max 40 edges (walking paths
    between sites)

    //Counter attributes for Graph class
    int vertexCount = 0;
    int edgeCount = 0;

    //Method to add a vertex to the graph
    public void addVertex(String name, int x, int y)
    {
        VertexList[vertexCount] = new Vertex(name, x, y);
        vertexCount++;
    }

    //Algorithm 1 - Search(site)
    // Algorithm to output details of a given site
    void Search(String site)
    {
        System.out.println("Search() output:");

        // Loop through all vertices to find the site
        for(int i = 0; i < vertexCount; i++)
        {
            // Check if the current vertex matches the site
            if(VertexList[i].name.equals(site))
            {
                System.out.println("Found site: " + site + " at coordinates
                (" + VertexList[i].x + ", " + VertexList[i].y + ")");
                return;
            }
        }
        // prints ONLY if site is not found
        System.out.println("Site not found in the graph.");
    }
}
```

```

//Algorithm 2 - Insert(String site1, String site2, double weight)
// Algorithm to insert a new edge between two sites with a given weight
void Insert(String site1, String site2, double weight)
{
    System.out.println("Insert() output:");

    // Create a new edge and add it to the EdgeList
    EdgeList[edgeCount] = new Edge(site1, site2, weight);
    edgeCount++;
    System.out.println("System inserted edge from " + site1 + " to " +
site2 + " with weight " + weight);
}

//Algorithm 3 - AllCons(String site)
// Algorithm to output names of all sites connected to the given site
void AllCons(String site)
{
    System.out.println("AllCons() output:");

    // Loops through all the edges to find the connection for any given
site
    System.out.println("Connections for: " + site);
    for(int i = 0; i < edgeCount; i++)
    {
        // Check if the current edge is connected to the site
        if (EdgeList[i].start.equals(site))
        {
            System.out.println("Connected to: " + EdgeList[i].end + "
with weight " + EdgeList[i].weight);
        }
        else if (EdgeList[i].end.equals(site))
        {
            System.out.println("Connected to: " + EdgeList[i].start + "
with weight " + EdgeList[i].weight);
        }
    }
}

//Algorithm 4 - Closest(site)
// Algorithm to find and output the closest connected site to the given
site
void Closest(String site)
{
    System.out.println("Closest() output:");

    // Variables to track the minimum weight and closest site
    double minWeight = Double.MAX_VALUE;
    String closestSite = null;

    // Loops through all edges to find the closest connected site
    for(int i = 0; i < edgeCount; i++)
    {
        // IF the current edge is connected to the site AND its weight
is less than the current minimum weight
        if (EdgeList[i].start.equals(site) && EdgeList[i].weight <
minWeight)
        {
            minWeight = EdgeList[i].weight;
            closestSite = EdgeList[i].end;
        }
    }
}

```

```

    }

    else if (EdgeList[i].end.equals(site) && EdgeList[i].weight <
minWeight)
    {
        minWeight = EdgeList[i].weight;
        closestSite = EdgeList[i].start;
    }
}
// Print the closest site and its weight
if (closestSite != null)
{
    System.out.println("Closest site to " + site + " is " +
closestSite + " with weight " + minWeight);
}
// prints if no connections found
else
{
    System.out.println("No connections found for site: " + site);
}
}
}

```